Section: 14. Functions

Lecture: 161. Math Functions - Built-in

Section 1: Lecture Summary

This lecture covers several **built-in Python functions related to mathematical operations** that programmers frequently use for computations. The instructor begins by listing functions such as `abs()`, `pow()`, `round()`, `divmod()`, `min()`, `max()`, `sum()`, and `eval()`. Each function is explained conceptually and demonstrated with examples in a Jupyter notebook context. Key points include understanding the input parameters (positional, keyword, and default arguments), how these functions behave with different types of inputs (integers, floats, and even complex numbers), and additional features like the `mod` argument for `pow()`, or the `key` and `default` arguments for `min()` and `max()`. The lecture particularly highlights Python's **banker's rounding** for the `round()` function and the power of `eval()` for evaluating expressions contained in strings along with custom global and local contexts.

Section 2: Key Concepts and Explanations

- **abs(x)**

  - Returns the absolute value (positive magnitude) of a number.

  - For real numbers (int, float), it returns the positive value.

  - For complex numbers \(a+bi\), it returns the magnitude: \(\sqrt{a^2 + b^2}\).

- **pow(base, exp, mod=None)**

  - Calculates `base` raised to the power `exp`.

  - If `exp` is negative, it returns the reciprocal (e.g., \(10^{-2} = 0.01\)).

  - An optional third argument `mod` can be provided to compute \((base^{exp}) \bmod mod\), useful to avoid large numbers in algorithms.

- **round(number, ndigits=None)**

  - Rounds `number` to the nearest integer if `ndigits` is not provided.

  - If `ndigits` is provided, rounds to that many decimal places.

  - Implements banker's rounding: when a number is exactly halfway between two integers, rounds to the nearest even number.

- Example: `round(4.5)` gives 4, `round(5.5)` gives 6.

- Can round to negative digits to round to tens, hundreds, etc.

- **divmod(a, b)**

- Returns a tuple `(quotient, remainder)` for integer division and modulus.

- Example: `divmod(10, 3)` returns `(3, 1)`.

- **min(iterable, , key=None, default=None)**

- Returns the smallest element in the iterable.

- `key` is a function applied to elements to determine comparison (e.g., `key=abs` compares absolute values).

- `default` is returned if iterable is empty instead of raising an error.

- **max(iterable, , key=None, default=None)**

- Similar to `min()`, returns the largest element.

- Can use `key` function for comparison, e.g., finding longest string with `key=len`.

- **sum(iterable, start=0)**

- Returns the sum of elements in iterable plus the optional `start` value (default 0).

- Example: `sum([1,2,3], start=10)` returns 16.

- **eval(expression, globals=None, locals=None)**

- Evaluates a string `expression` as a Python expression.

- Can take `globals` and `locals` dictionaries providing variable context for evaluation.

- Useful to dynamically evaluate or compute expressions from strings safely within controlled namespaces.

Section 3: Example Code and Use Cases

abs examples

```
print(abs(-70))           # Output: 70
print(abs(-70.12))        # Output: 70.12
print(abs(3 + 4j))        # Output: 5.0 (magnitude of complex number)
```

Returns absolute values and magnitude for complex numbers.

pow examples

```
print(pow(2, 3))          # Output: 8 (2^3)
print(pow(10, 2))         # Output: 100 (10^2)
print(pow(10, -2))        # Output: 0.01 (1/10^2)
print(pow(10, 2, 3))      # Output: 1 (100 mod 3)
```

Calculates power with optional modulo for large numbers.

round examples

```
print(round(4.4))          # Output: 4
print(round(4.6))          # Output: 5
print(round(4.5))          # Output: 4 (banker's rounding)
print(round(5.5))          # Output: 6 (banker's rounding)
print(round(3.54321, 2))   # Output: 3.54 (round to 2 decimal places)
print(round(9734, -1))     # Output: 9730 (round to nearest tens)
print(round(9734, -2))     # Output: 9700 (round to nearest hundreds)
```

Demonstrates banker's rounding and rounding with decimal and negative digits.

divmod examples

```
print(divmod(10, 3))       # Output: (3, 1) -> quotient=3, remainder=1
print(divmod(61, 7))       # Output: (8, 5)
```

Returns quotient and remainder in a tuple.

min with key and default arguments

```
numbers = [-10, 3, 7, -2, -5]
print(min(numbers))                # Output: -10
print(min(numbers, key=abs))      # Output: -2 (minimum by absolute value)
print(min([], default="Empty"))   # Output: 'Empty' (no error on empty
iterable)
```

Finds minimum with conditions.

max with key argument

```
words = ['apple', 'banana', 'cherry', 'blueberry']
print(max(words, key=len))        # Output: 'blueberry' (longest word)
```

Finds maximum element based on length.

sum examples

```
print(sum([1, 2, 3, 4, 5]))             # Output: 15
print(sum([1, 2, 3, 4, 5], start=20))  # Output: 35 (20 + 15)
```

Sums iterable values with an optional starting offset.

eval examples

```
globals_dict = {"x": 10, "y": 20}
locals_dict = {"z": 5}
expr = "x + y + z"
print(eval(expr, globals_dict, locals_dict))  # Output: 35
```

Evaluates expression strings dynamically with variable context dictionaries.

Section 4: Key Takeaways

- Python provides numerous **built-in mathematical functions** to simplify common tasks such as absolute value, power, rounding, division with remainder, minimum & maximum search, summation, and expression evaluation.

- Functions can accept **positional, keyword, and default arguments** to provide flexibility and control.

- `abs()` works for integers, floats, and complex numbers (returning magnitude).

- `pow()` supports an optional modulus operand for modular exponentiation, useful in algorithmic problems.

- `round()` uses **banker's rounding**, rounding to the nearest even integer when exactly halfway.

- `divmod()` efficiently returns quotient and remainder as a tuple in a single call.

- `min()` and `max()` accept `key` functions for customized comparisons and `default` inputs to avoid errors on empty iterables.

- `sum()` can begin with a non-zero `start` value.

- `eval()` can evaluate Python expressions given in string form, utilizing optional globals and locals dictionaries for safe variable context.

- Understanding these built-in functions and their arguments can improve code clarity, efficiency, and robustness in mathematical calculations.